# 10.6. Synchronization Functions

```
void __syncthreads();
```

waits until all threads in the thread block have reached this point and all global and shared memory accesses made by these threads prior to `__syncthreads()` are visible to all threads in the block.

`__syncthreads()` is used to coordinate communication between the threads of the same block. When some threads within a block access the same addresses in shared or global memory, there are potential read-after-write, write-after-read, or write-after-write hazards for some of these memory accesses. These data hazards can be avoided by synchronizing threads in-between these accesses.

`__syncthreads()` is allowed in conditional code but only if the conditional evaluates identically across the entire thread block, otherwise the code execution is likely to hang or produce unintended side effects.

Devices of compute capability 2.x and higher support three variations of `__syncthreads()` described below.

```
int __syncthreads_count(int predicate);
```

is identical to `__syncthreads()` with the additional feature that it evaluates predicate for all threads of the block and returns the number of threads for which predicate evaluates to non-zero.

```
int __syncthreads_and(int predicate);
```

is identical to `__syncthreads()` with the additional feature that it evaluates predicate for all threads of the block and returns non-zero if and only if predicate evaluates to non-zero for all of them.

```
int __syncthreads_or(int predicate);
```

is identical to `__syncthreads()` with the additional feature that it evaluates predicate for all threads of the block and returns non-zero if and only if predicate evaluates to non-zero for any of them.

```
void __syncwarp(unsigned mask=0xffffffff);
```

will cause the executing thread to wait until all warp lanes named in mask have executed a `__syncwarp()` (with the same mask) before resuming execution. Each calling thread must have its own bit set in the mask and all non-exited threads named in mask must execute a corresponding `__syncwarp()` with the same mask, or the result is undefined.

Executing `__syncwarp()` guarantees memory ordering among threads participating in the barrier. Thus, threads within a warp that wish to communicate via memory can store to memory, execute `__syncwarp()`, and then safely read values stored by other threads in the warp.

---

**Note:** For .target sm_6x or below, all threads in mask must execute the same `__syncwarp()` in convergence, and the union of all values in mask must be equal to the active mask. Otherwise, the behavior is undefined.

---

(continued from previous page)

```
                                __longlong_as_double(assumed)));

    // Note: uses integer comparison to avoid hang in case of NaN (since NaN != NaN)
    } while (assumed != old);

    return __longlong_as_double(old);
}
#endif
```

There are system-wide and block-wide variants of the following device-wide atomic APIs, with the following exceptions:

- Devices with compute capability less than 6.0 only support device-wide atomic operations,
- Tegra devices with compute capability less than 7.2 do not support system-wide atomic operations.
- The system-wide scope is not supported for 128-bit variants of `atomicCAS()` and `atomicExch()`.

## 10.14.1. Arithmetic Functions

#### 10.14.1.1 atomicAdd()

```
int atomicAdd(int* address, int val);
unsigned int atomicAdd(unsigned int* address,
                       unsigned int val);
unsigned long long int atomicAdd(unsigned long long int* address,
                                 unsigned long long int val);
float atomicAdd(float* address, float val);
double atomicAdd(double* address, double val);
__half2 atomicAdd(__half2 *address, __half2 val);
__half atomicAdd(__half *address, __half val);
__nv_bfloat162 atomicAdd(__nv_bfloat162 *address, __nv_bfloat162 val);
__nv_bfloat16 atomicAdd(__nv_bfloat16 *address, __nv_bfloat16 val);
float2 atomicAdd(float2* address, float2 val);
float4 atomicAdd(float4* address, float4 val);
```

reads the 16-bit, 32-bit or 64-bit `old` located at the address `address` in global or shared memory, computes `(old + val)`, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns `old`.

The 32-bit floating-point version of `atomicAdd()` is only supported by devices of compute capability 2.x and higher.

The 64-bit floating-point version of `atomicAdd()` is only supported by devices of compute capability 6.x and higher.

The 32-bit `__half2` floating-point version of `atomicAdd()` is only supported by devices of compute capability 6.x and higher. The atomicity of the `__half2` or `__nv_bfloat162` add operation is guaranteed separately for each of the two `__half` or `__nv_bfloat16` elements; the entire `__half2` or `__nv_bfloat162` is not guaranteed to be atomic as a single 32-bit access.

The `float2` and `float4` floating-point vector versions of `atomicAdd()` are only supported by devices of compute capability 9.x and higher. The atomicity of the `float2` or `float4` add operation is guar-

anteed separately for each of the two or four `float` elements; the entire `float2` or `float4` is not guaranteed to be atomic as a single 64-bit or 128-bit access.

The 16-bit `__half` floating-point version of `atomicAdd()` is only supported by devices of compute capability 7.x and higher.

The 16-bit `__nv_bfloat16` floating-point version of `atomicAdd()` is only supported by devices of compute capability 8.x and higher.

The `float2` and `float4` floating-point vector versions of `atomicAdd()` are only supported by devices of compute capability 9.x and higher.

The `float2` and `float4` floating-point vector versions of `atomicAdd()` are only supported for global memory addresses.

#### 10.14.1.2 atomicSub()

```
int atomicSub(int* address, int val);
unsigned int atomicSub(unsigned int* address,
                       unsigned int val);
```

reads the 32-bit word `old` located at the address `address` in global or shared memory, computes `(old - val)`, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns `old`.

#### 10.14.1.3 atomicExch()

```
int atomicExch(int* address, int val);
unsigned int atomicExch(unsigned int* address,
                        unsigned int val);
unsigned long long int atomicExch(unsigned long long int* address,
                                  unsigned long long int val);
float atomicExch(float* address, float val);
```

reads the 32-bit or 64-bit word `old` located at the address `address` in global or shared memory and stores `val` back to memory at the same address. These two operations are performed in one atomic transaction. The function returns `old`.

```
template<typename T> T atomicExch(T* address, T val);
```

reads the 128-bit word `old` located at the address `address` in global or shared memory and stores `val` back to memory at the same address. These two operations are performed in one atomic transaction. The function returns `old`. The type T must meet the following requirements:

```
sizeof(T) == 16
alignof(T) >= 16
std::is_trivially_copyable<T>::value == true
// for C++03 and older
std::is_default_constructible<T>::value == true
```

So, T must be 128-bit and properly aligned, be trivially copyable, and on C++03 or older, it must also be default constructible.

The 128-bit `atomicExch()` is only supported by devices of compute capability 9.x and higher.

#### 10.14.1.4 atomicMin()

```
int atomicMin(int* address, int val);
unsigned int atomicMin(unsigned int* address,
                       unsigned int val);
unsigned long long int atomicMin(unsigned long long int* address,
                                 unsigned long long int val);
long long int atomicMin(long long int* address,
                                long long int val);
```

reads the 32-bit or 64-bit word `old` located at the address `address` in global or shared memory, computes the minimum of `old` and `val`, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns `old`.

The 64-bit version of `atomicMin()` is only supported by devices of compute capability 5.0 and higher.

#### 10.14.1.5 atomicMax()

```
int atomicMax(int* address, int val);
unsigned int atomicMax(unsigned int* address,
                       unsigned int val);
unsigned long long int atomicMax(unsigned long long int* address,
                                 unsigned long long int val);
long long int atomicMax(long long int* address,
                                 long long int val);
```

reads the 32-bit or 64-bit word `old` located at the address `address` in global or shared memory, computes the maximum of `old` and `val`, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns `old`.

The 64-bit version of `atomicMax()` is only supported by devices of compute capability 5.0 and higher.

#### 10.14.1.6 atomicInc()

```
unsigned int atomicInc(unsigned int* address,
                       unsigned int val);
```

reads the 32-bit word `old` located at the address `address` in global or shared memory, computes `((old >= val) ? 0 : (old+1))`, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns `old`.

#### 10.14.1.7 atomicDec()

```
unsigned int atomicDec(unsigned int* address,
                       unsigned int val);
```

reads the 32-bit word `old` located at the address `address` in global or shared memory, computes `(((old == 0) || (old > val)) ? val : (old-1) )`, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns `old`.

#### 10.14.1.8 atomicCAS()

```
int atomicCAS(int* address, int compare, int val);
unsigned int atomicCAS(unsigned int* address,
                       unsigned int compare,
                       unsigned int val);
unsigned long long int atomicCAS(unsigned long long int* address,
                                 unsigned long long int compare,
                                 unsigned long long int val);
unsigned short int atomicCAS(unsigned short int *address,
                             unsigned short int compare,
                             unsigned short int val);
```

reads the 16-bit, 32-bit or 64-bit word `old` located at the address `address` in global or shared memory, computes `(old == compare ? val : old)`, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns `old` (Compare And Swap).

```
template<typename T> T atomicCAS(T* address, T compare, T val);
```

reads the 128-bit word `old` located at the address `address` in global or shared memory, computes `(old == compare ? val : old)`, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns `old` (Compare And Swap). The type T must meet the following requirements:

```
sizeof(T) == 16
alignof(T) >= 16
std::is_trivially_copyable<T>::value == true
// for C++03 and older
std::is_default_constructible<T>::value == true
```

So, T must be 128-bit and properly aligned, be trivially copyable, and on C++03 or older, it must also be default constructible.

The 128-bit `atomicCAS()` is only supported by devices of compute capability 9.x and higher.

## 10.14.2. Bitwise Functions

#### 10.14.2.1 atomicAnd()

```
int atomicAnd(int* address, int val);
unsigned int atomicAnd(unsigned int* address,
                       unsigned int val);
unsigned long long int atomicAnd(unsigned long long int* address,
                                 unsigned long long int val);
```

reads the 32-bit or 64-bit word `old` located at the address `address` in global or shared memory, computes `(old & val)`, and stores the result back to memory at the same address. These three operations are performed in one atomic transaction. The function returns `old`.

The 64-bit version of `atomicAnd()` is only supported by devices of compute capability 5.0 and higher.